

AI SOFTWARE DEVELOPMENT INTERNSHIP

EMAN MASOOD (BSSE)

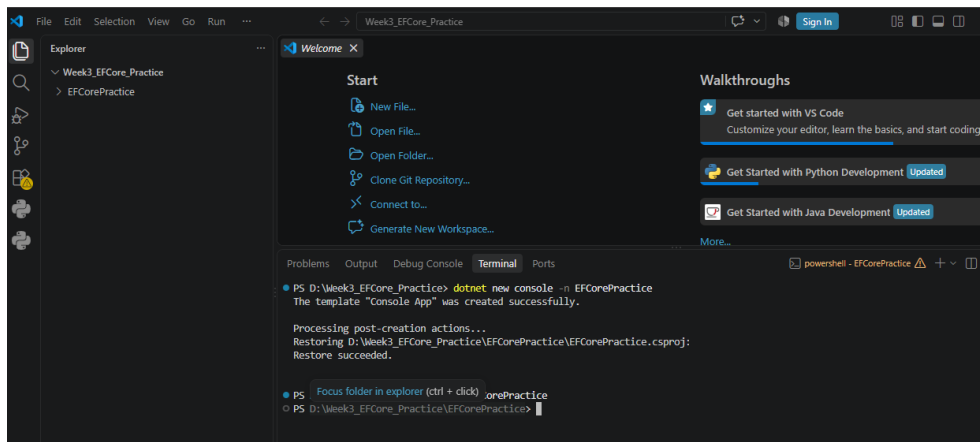
Week 3: SQL Server & Entity Framework Core • API/Angular
Integration Git & GitHub Workflow • AI & Python Foundations

PART B — ENTITY FRAMEWORK CORE

Practice Tasks

1. Create a small standalone EF Core console/API project with just one entity to get comfortable with the migration commands before touching the real Library project.

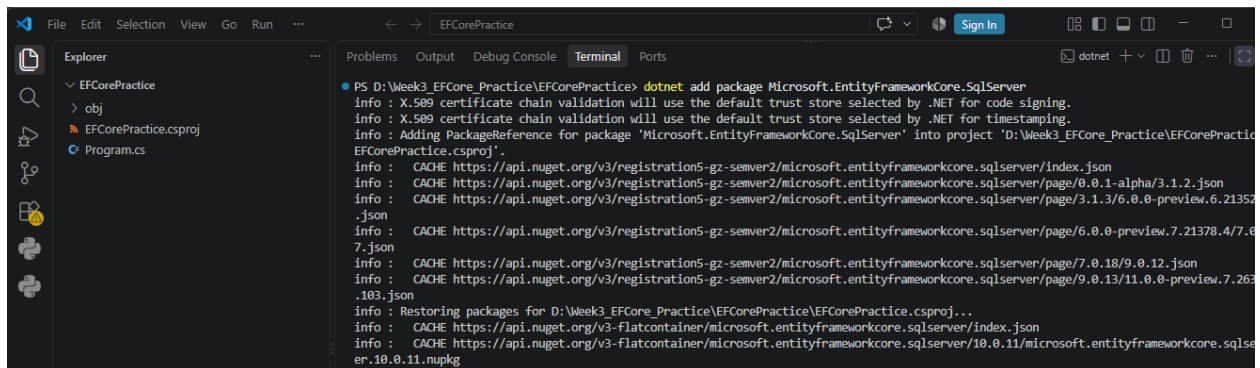
Output:



This screenshot shows the Visual Studio Code interface with a new console application created. The Explorer pane on the left shows the project structure: Week3_EFCore_Practice > EFCorePractice. The Start pane in the center offers options like 'New File...', 'Open File...', and 'Generate New Workspace...'. The Terminal pane at the bottom shows the command `dotnet new console -n EFCorePractice` being executed, with output indicating the template was created successfully and packages were restored.

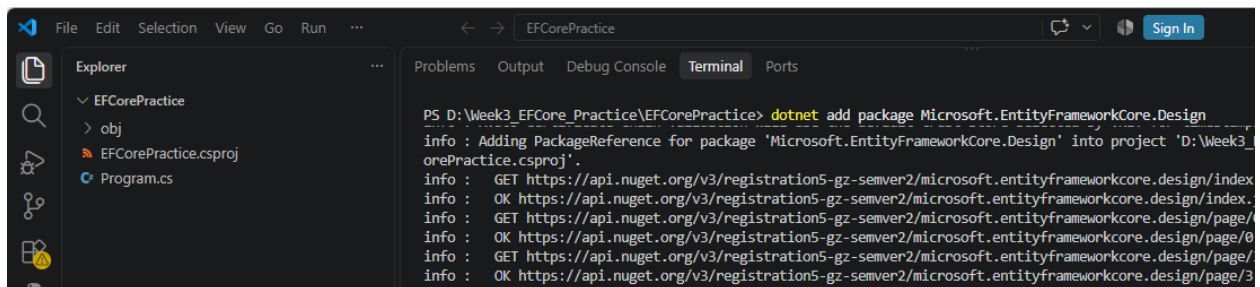
```
PS D:\Week3_EFCore_Practice> dotnet new console -n EFCorePractice
The template "Console App" was created successfully.

Processing post-creation actions...
Restoring D:\Week3_EFCore_Practice\EFCorePractice\EFCorePractice.csproj:
Restore succeeded.
```



This screenshot shows the Visual Studio Code interface with the `dotnet add package` command being executed. The Explorer pane shows the project structure: EFCorePractice > obj > EFCorePractice.csproj > Program.cs. The Terminal pane shows the command `dotnet add package Microsoft.EntityFrameworkCore.SqlServer` and its output, which includes information about adding the package reference and restoring packages from the NuGet repository.

```
PS D:\Week3_EFCore_Practice\EFCorePractice> dotnet add package Microsoft.EntityFrameworkCore.SqlServer
info : X.509 certificate chain validation will use the default trust store selected by .NET for code signing.
info : X.509 certificate chain validation will use the default trust store selected by .NET for timestamping.
info : Adding PackageReference for package 'Microsoft.EntityFrameworkCore.SqlServer' into project 'D:\Week3_EFCore_Practice\EFCorePractice\EFCorePractice.csproj'.
info : CACHE https://api.nuget.org/v3/registrations5-gz-semver2/microsoft.entityframeworkcore.sqlserver/index.json
info : CACHE https://api.nuget.org/v3/registrations5-gz-semver2/microsoft.entityframeworkcore.sqlserver/page/0.0.1-alpha/3.1.2.json
info : CACHE https://api.nuget.org/v3/registrations5-gz-semver2/microsoft.entityframeworkcore.sqlserver/page/3.1.3/6.0.0-preview.6.21352.json
info : CACHE https://api.nuget.org/v3/registrations5-gz-semver2/microsoft.entityframeworkcore.sqlserver/page/6.0.0-preview.7.21378.4/7.0.0-preview.8.21404.12.json
info : CACHE https://api.nuget.org/v3/registrations5-gz-semver2/microsoft.entityframeworkcore.sqlserver/page/7.0.18/9.0.12.json
info : CACHE https://api.nuget.org/v3/registrations5-gz-semver2/microsoft.entityframeworkcore.sqlserver/page/9.0.13/11.0.0-preview.7.263103.json
info : Restoring packages for D:\Week3_EFCore_Practice\EFCorePractice\EFCorePractice.csproj...
info : CACHE https://api.nuget.org/v3-flatcontainer/microsoft.entityframeworkcore.sqlserver/index.json
info : CACHE https://api.nuget.org/v3-flatcontainer/microsoft.entityframeworkcore.sqlserver/10.0.11/microsoft.entityframeworkcore.sqlserver.10.0.11.nupkg
info : OK https://api.nuget.org/v3-flatcontainer/microsoft.entityframeworkcore.sqlserver/10.0.11/microsoft.entityframeworkcore.sqlserver.10.0.11.nupkg
```



This screenshot shows the Visual Studio Code interface with the `dotnet add package` command being executed. The Explorer pane shows the project structure: EFCorePractice > obj > EFCorePractice.csproj > Program.cs. The Terminal pane shows the command `dotnet add package Microsoft.EntityFrameworkCore.Design` and its output, which includes information about adding the package reference and restoring packages from the NuGet repository.

```
PS D:\Week3_EFCore_Practice\EFCorePractice> dotnet add package Microsoft.EntityFrameworkCore.Design
info : Adding PackageReference for package 'Microsoft.EntityFrameworkCore.Design' into project 'D:\Week3_EFCore_Practice\EFCorePractice.csproj'.
info : GET https://api.nuget.org/v3/registrations5-gz-semver2/microsoft.entityframeworkcore.design/index.json
info : OK https://api.nuget.org/v3/registrations5-gz-semver2/microsoft.entityframeworkcore.design/index.json
info : GET https://api.nuget.org/v3/registrations5-gz-semver2/microsoft.entityframeworkcore.design/page/0.0.1-alpha/3.1.2.json
info : OK https://api.nuget.org/v3/registrations5-gz-semver2/microsoft.entityframeworkcore.design/page/0.0.1-alpha/3.1.2.json
info : GET https://api.nuget.org/v3/registrations5-gz-semver2/microsoft.entityframeworkcore.design/page/3.1.3/6.0.0-preview.6.21352.json
info : OK https://api.nuget.org/v3/registrations5-gz-semver2/microsoft.entityframeworkcore.design/page/3.1.3/6.0.0-preview.6.21352.json
```

```

info : CAQHE https://api.nuget.org/v3/vulnerabilities/index.json
info : CAQHE https://api.nuget.org/v3/vulnerabilities/2026.08.27.05.48.31/vulnerability.base.json
info : CAQHE https://api.nuget.org/v3/vulnerabilities/2026.08.27.05.48.31/2026.09.01.11.49.02/vulnerabi
info : Package 'Microsoft.EntityFrameworkCore.Design' is compatible with all the specified frameworks in
EFCorePractice\EFCorePractice.csproj".
info : PackageReference for package 'Microsoft.EntityFrameworkCore.Design' version '10.0.11' added to fil
Practice\EFCorePractice.csproj".
info : Generating MSBuild file D:\Week3_EFCore_Practice\EFCorePractice\obj\EFCorePractice.csproj.nuget.g
info : Generating MSBuild file D:\Week3_EFCore_Practice\EFCorePractice\obj\EFCorePractice.csproj.nuget.g
info : Writing assets file to disk. Path: D:\Week3_EFCore_Practice\EFCorePractice\obj\project.assets.json
log : Restored D:\Week3_EFCore_Practice\EFCorePractice\EFCorePractice.csproj (in 1.92 min).
● PS D:\Week3_EFCore_Practice\EFCorePractice> dotnet tool install --global dotnet-ef
You can invoke the tool using the following command: dotnet-ef
Tool 'dotnet-ef' (version '10.0.11') was successfully installed.
● PS D:\Week3_EFCore_Practice\EFCorePractice> dotnet ef --version
Entity Framework Core .NET Command-line Tools
10.0.11
○ PS D:\Week3_EFCore_Practice\EFCorePractice>

```

The screenshot shows the Visual Studio IDE with the EFCorePractice project open. The Explorer pane on the left displays the project structure, including folders for bin, Data, LibraryDbContext, Models, Books, obj, and Program.cs. The Program.cs file is selected and its content is shown in the main editor. The code defines a DbContext, a DbContextOptionsBuilder, and a Program class that sets up the database context and runs the application.

```

1 using EFCorePractice.data;
2 using Microsoft.EntityFrameworkCore;
3
4 var options = new DbContextOptionsBuilder<LibraryDbContext>()
5     .UseSqlServer(
6         "Server=localhost;Database=EFCorePracticeDb;Trusted_Connection=True;TrustServerCertificate=True;"
7     ).Options;
8
9 using var context = new LibraryDbContext(options);
10
11 Console.WriteLine("EF Core Practice Project is Ready!");

```

The Output pane at the bottom shows the build and execution logs, including the message "Build succeeded in 17.9s" and the command to run the application: `bin\Debug\net10.0\EFCorePractice.dll`.

The screenshot shows the Visual Studio IDE with the following components:

- File Explorer (Left):** Displays the project structure for `EFCorePractice`. The `LibraryDbContextFactory.cs` file is highlighted under the `LibraryDbContextFactory.cs` folder.
- Code Editor (Center):** Shows the content of `LibraryDbContextFactory.cs`. The code defines a `DbContextFactory` class that implements `IDesignTimeDbContextFactory<LibraryDbContext>`. It includes a `CreateDbContext` method that returns a new `LibraryDbContext` instance.
- Terminal (Bottom):** Shows the output of the command `dotnet build` and the command `dotnet ef migrations add InitialCreate`. The output indicates that the build was successful and that the migration was added.

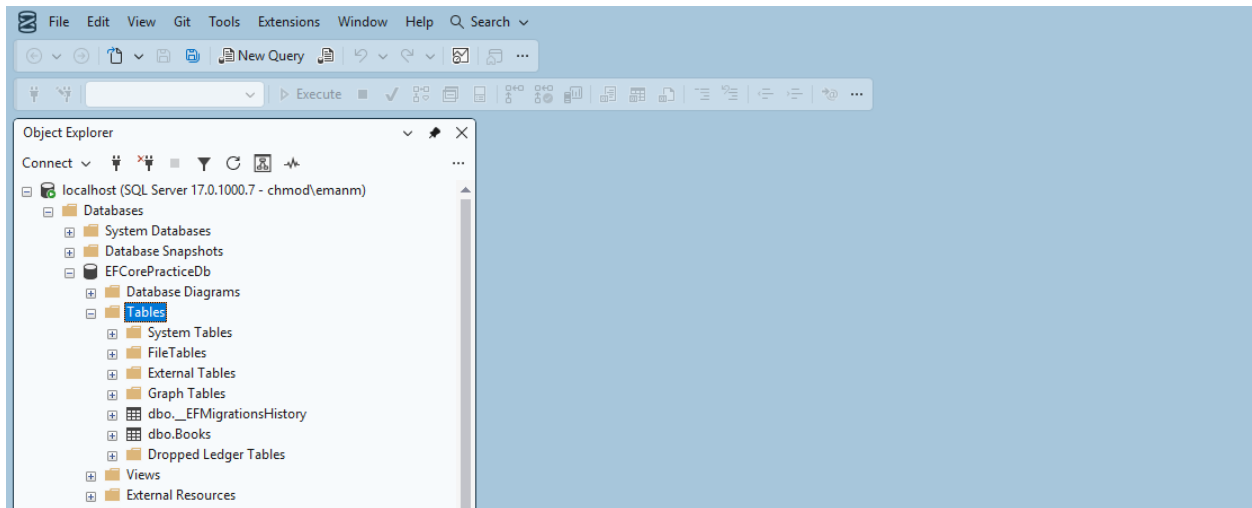
```
> obj
EFCorePractice.csproj
Program.cs

PS D:\Week3_EFCore_Practice\EFCorePractice> dotnet build
Microsoft (R) Build Engine
Build succeeded.
0 Warning(s)
0 Error(s)

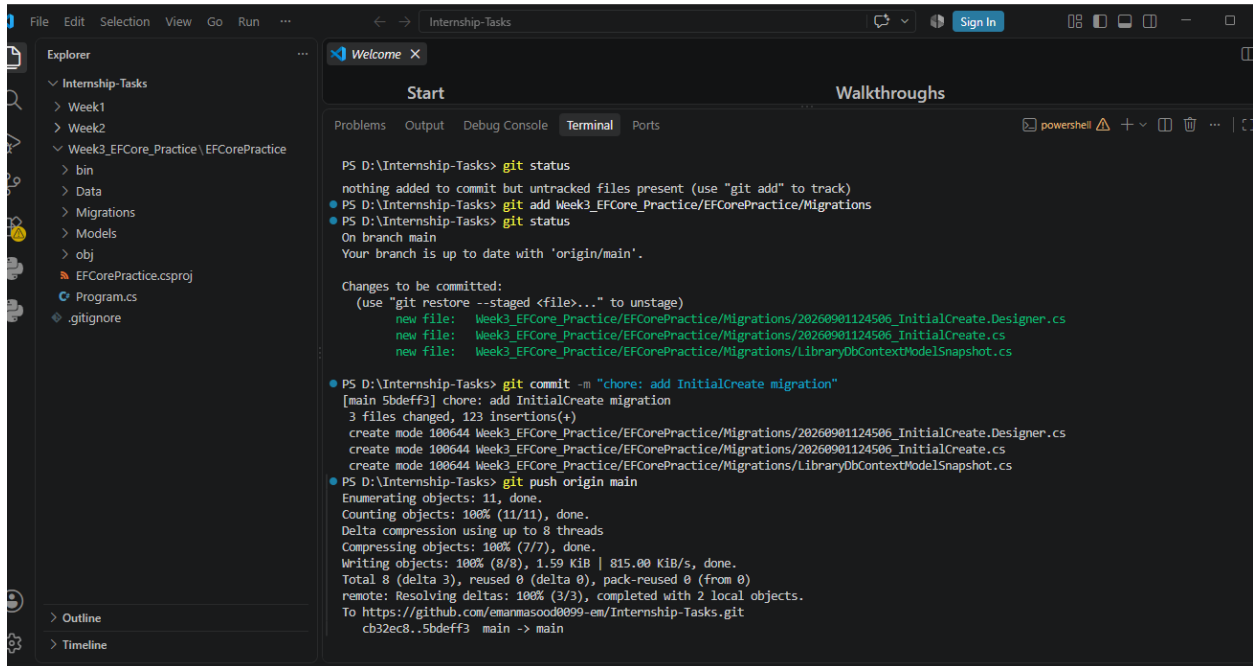
PS D:\Week3_EFCore_Practice\EFCorePractice> dotnet ef migrations add InitialCreate
Build started...
Build succeeded.
Done. To undo this action, use 'ef migrations remove'

PS D:\Week3_EFCore_Practice\EFCorePractice> dotnet ef database update
Build started...
Build succeeded.
Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-migrations-long.
Applying migration '20260901124506_InitialCreate'.
Done.

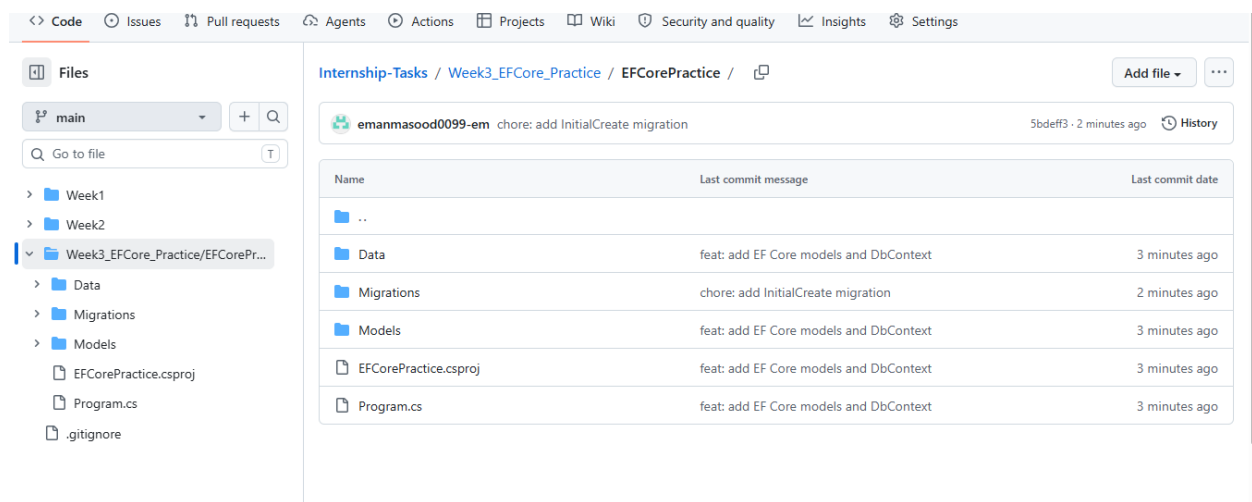
PS D:\Week3_EFCore_Practice\EFCorePractice> 
```



Git Checkpoint Commit after the migration is created and again after it's applied, so the history shows the steps clearly: `git commit -m "feat: add EF Core models and DbContext"` then `git commit -m "chore: add InitialCreate migration"`

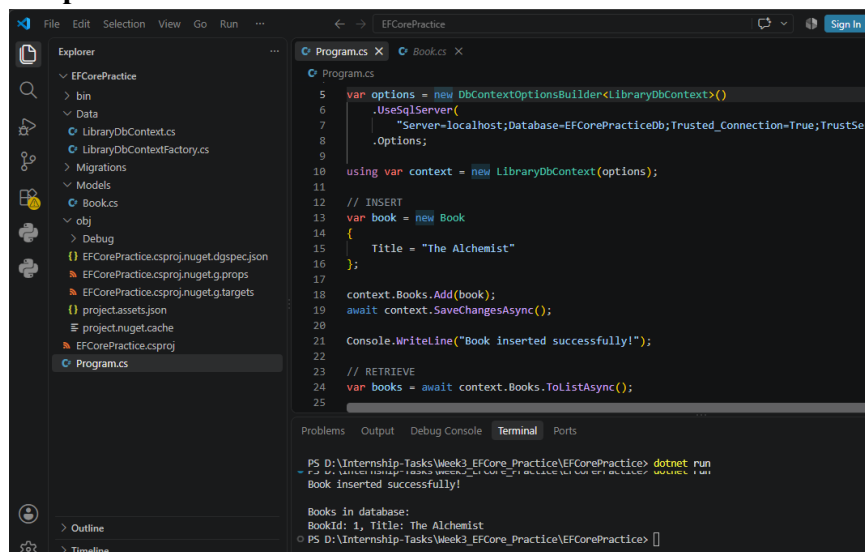


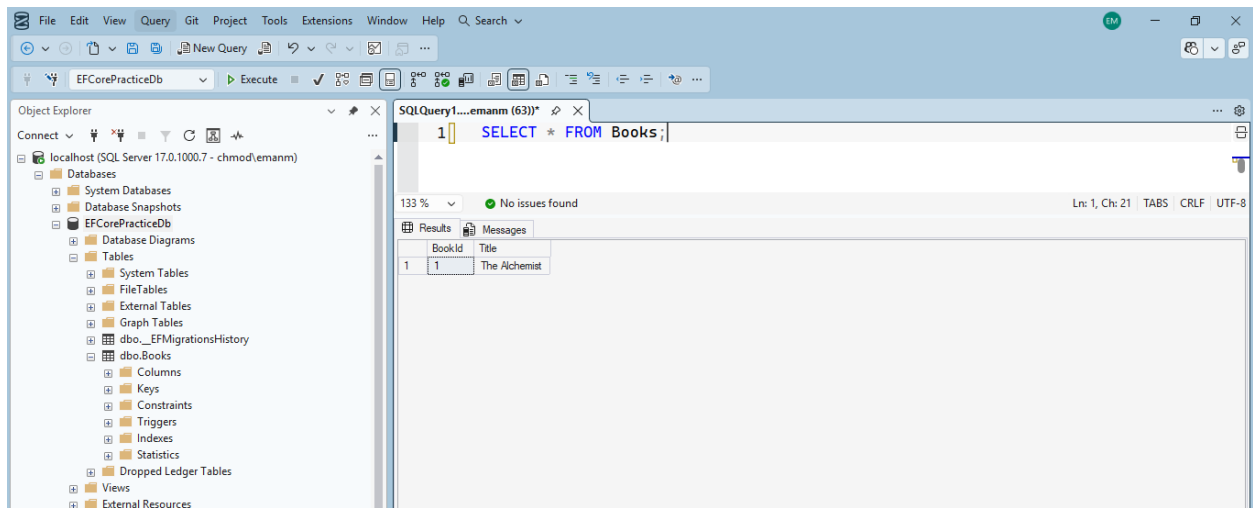
```
Writing objects: 100% (8/8), 1.59 KiB | 815.00 KiB/s, done.
Total 8 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 2 local objects.
To https://github.com/emanmasood0099-em/Internship-Tasks.git
   cb32ec8..5bdeff3  main -> main
● PS D:\Internship-Tasks> git log --oneline --all
5bdeff3 (HEAD -> main, origin/main) chore: add InitialCreate migration
cb32ec8 feat: add EF Core models and DbContext
068ed62 Add Week 1 and Week 2 internship tasks
○ PS D:\Internship-Tasks> █
```



2. Insert and retrieve a few records using context.Books.Add(...) and await context.SaveChangesAsync().

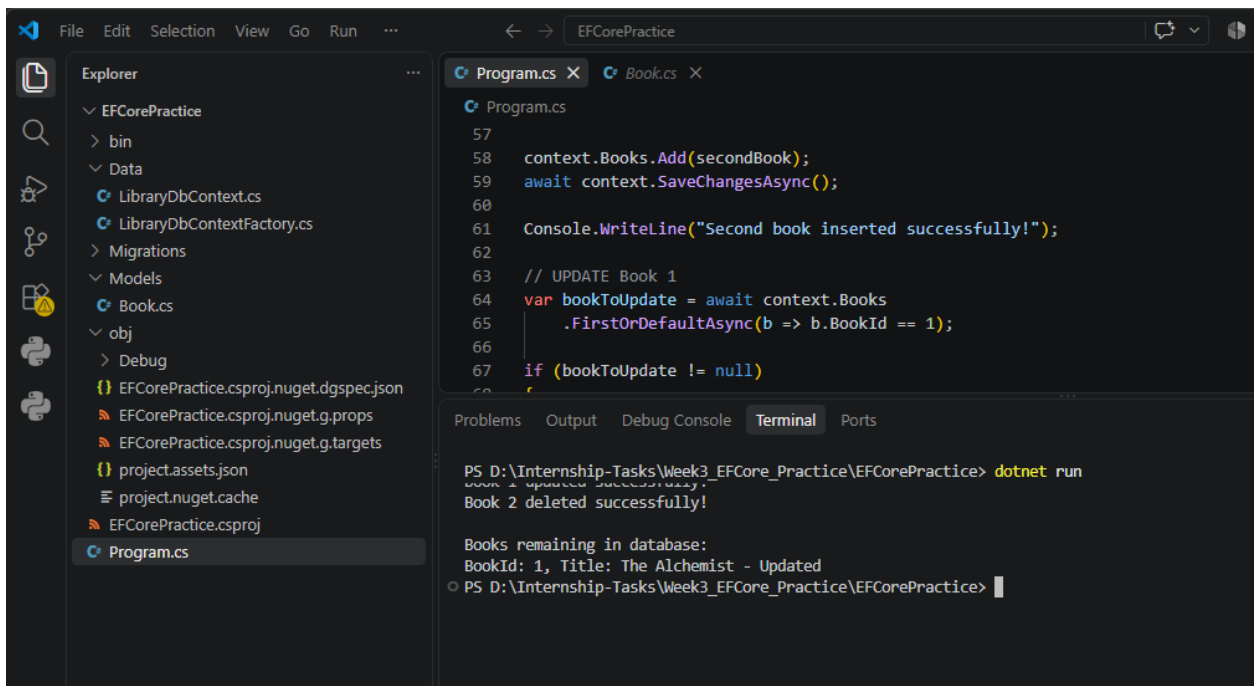
Output:

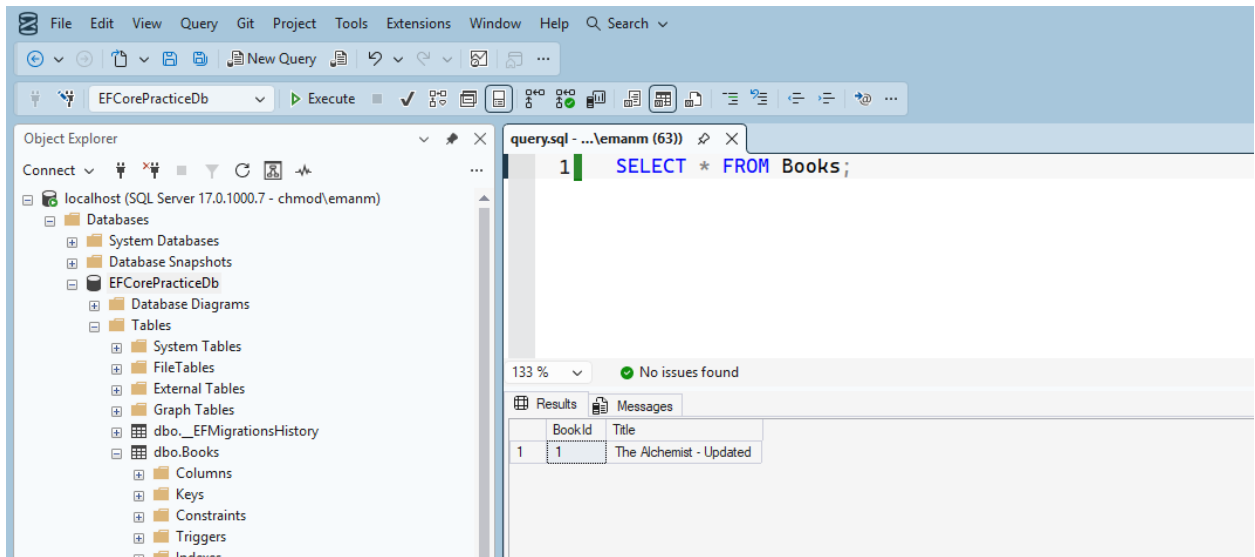




3. Update one record and delete another using EF Core (not raw SQL).

Output:





4. Write one filtered LINQ query against the DbSet, e.g. `context.Books.Where(b => b.AuthorId == 2).ToListAsync()`.

Output:

